

Cargoservice — Sprint1

ISS 2026 — VALERIA CAPACCI & GINEVRA PENTOLI

[GOAL](#) [PROBLEM ANALYSIS](#) [TEST PLANS](#) [PROJECT](#) [DEPLOYMENT](#) [SPRINT2](#)

Goal

Realizzare il sottosistema che gestisce l'accettazione/rifiuto di una richiesta di carico, il timeout dei 30 secondi, e l'interazione reale con l' `IOPort` . `cargoservice` riceve `loadrequest` , interroga `Hold` , risponde con `loadaccepted` / `loadrejected` e transita da `Idle` a `Engaged`; se il timeout scade torna a `Idle` liberando lo slot.

Problem analysis

Individuazione dei componenti necessari

`cargoservice` deve reagire a vari tipi di richieste: deve rispondere sia alla pressione del pulsante sia al semplice scadere del tempo, nessuna delle due cosa ottenibile da un oggetto passivo. Per questo motivo, scegliamo di rappresentarlo come un `QActor`, come già discusso precedentemente nello Sprint0. Per lo stesso motivo `ioport` , che gestisce l'interazione con il cliente, è anch'esso un attore: riceve stimoli da due direzioni indipendenti: invia notifiche da parte del cliente (es. pulsante premuto) e riceve (e gestisce) le notifiche inviate da `cargoservice` (es. esito della `loadrequest`).

A questi componenti essenziali occorre poi analizzare e valutare se è necessario aggiungerne altri.

Scelta della tecnologia di implementazione dell'IOPort

L'IOPort è un nodo architetturale a sé stante rispetto a `cargoservice` : comunica con l'utente finale e la sua implementazione non deve necessariamente ricalcare lo stack tecnologico del resto del sistema. Le alternative concretamente valutate per realizzarlo sono state un pannello embedded con touchscreen e firmware proprio, oppure una WebGUI fruibile da browser. Il committente ha indicato una preferenza esplicita per la seconda. La scelta resta comunque coerente: rispetto al pannello, riduce i costi di manutenzione e permette che lo stato della hold sia visualizzabile da più utenti contemporaneamente, senza vincolare la soluzione a un singolo dispositivo fisico.

Il costo di questa scelta è una disomogeneità tecnologica che il pannello embedded non avrebbe introdotto: mentre `cargoservice` e `ioport` vivono nello stack Kotlin/qak, l'interfaccia utente è codice JavaScript/HTML in esecuzione nel browser, un ambiente che, come discusso di seguito, non parla lo stesso protocollo né lo stesso linguaggio del resto del sistema. Questa eterogeneità viene quindi accettata come costo del requisito multi-utente.

Perché il canale interno di qak non basta e perché non si fa polling

Qak usa TCP come trasporto tra `Context` : ogni `Context` apre una socket in ascolto, e il runtime serializza/deserializza gli `IAppLMessage` scambiati tra attori in un formato interno, noto solo alle due estremità qak-qak. Un browser non può costruire né interpretare questo formato, né, più in generale, aprire un socket TCP grezzo: per motivi di sicurezza, il sandboxing dei browser espone a JavaScript solo un insieme ristretto di API di rete (richieste HTTP/HTTPS, WebSocket, e ciò che vi si costruisce sopra), non l'accesso diretto a socket TCP o UDP. È per questo che anche CoAP, che si appoggia su UDP, non è utilizzabile da una pagina web. MQTT è invece raggiungibile da browser solo se il broker espone un bridge MQTT-over-WsWebSocket: il vero canale attraversato resta comunque una WebSocket.

Il canale verso il browser va quindi progettato a parte, e deve soddisfare un requisito di aggiornamento in tempo reale: più utenti devono vedere lo stato della hold e l'esito delle richieste non appena cambia. Una prima

alternativa scartata è il polling HTTP (il client richiede periodicamente lo stato): introduce latenza tra il cambiamento di stato e la sua visualizzazione, e un carico crescente con il numero di client connessi che interroga il server anche quando nulla è cambiato, inadeguato a un pannello che più clienti potrebbero osservare contemporaneamente. Restano da confrontare i due protocolli effettivamente push e raggiungibili da un browser:

Alternativa	Limite rispetto al caso d'uso
MQTT (via bridge WebSocket)	modello publish/subscribe, richiede un broker come componente architetturale aggiuntivo; la bidirezionalità punto-a-punto richiesta qui (comando dal client, risposta al sistema) è ottenibile solo simulandola con due topic distinti
WebSocket	canale full-duplex nativo su una singola connessione TCP, senza componenti architetturali aggiuntivi: il server emette un aggiornamento non appena pronto, senza che il client debba richiederlo

La scelta ricade su WebSocket proprio perché non introduce un broker: la comunicazione avviene direttamente tra client e server applicativo.

Formato dei messaggi

L'infrastruttura qak sa già trasformare una stringa ricevuta su una connessione in un messaggio applicativo (`IAppLMessage`): è esattamente ciò che fa quando due attori comunicano via TCP. A rigore, quindi, non è la traduzione in sé il problema: se il browser inviasse messaggi già nello stesso formato riconosciuto dall'infrastruttura, servirebbe solo un componente che li accolga sulla connessione WebSocket e li consegna alla coda dell'attore, senza alcuna conversione.

Ciò che l'analisi deve garantire è che le due estremità del canale (la pagina web e `ioport`) si accordino su un formato condiviso in cui rappresentare i comandi che la GUI può inviare. Se questo formato condiviso coincide con quello nativo di qak non serve alcuna conversione; se invece si adotta, per il lato browser, un vocabolario applicativo più minimale, diventa necessario un passo di mappatura tra i due.

Il componente ponte tra WebSocket e attore

Il protocollo WebSocket introduce un vincolo indipendente dal formato dei messaggi: la libreria che lo implementa gestisce ogni connessione sul proprio thread, distinto da quello su cui gira il ciclo di esecuzione di `ioport` . Il modello ad attori di qak richiede che lo stato di un attore sia modificato esclusivamente dal thread dell'attore stesso, in risposta ai messaggi sulla propria coda: nessun altro thread può invocare direttamente metodi che ne alterino lo stato. Un messaggio WebSocket in arrivo sul thread del server non può quindi tradursi in una chiamata diretta su `ioport` . Serve dunque un componente dedicato che riceva l'evento sul proprio thread e lo consegna alla coda dell'attore tramite `sendMsgToMyself` , l'unico canale ammesso. Questo componente, `IOPortWsAdapter` , non è un attore qak: è un oggetto Java di proprietà esclusiva di `ioport` , che lo istanzia alla propria creazione passandogli un riferimento a se stesso. Esso vive interamente dentro il nodo `ioport` e non comunica in alcun modo, né diretto né indiretto, con `cargoservice` o con altri attori: è `ioport` , come già discusso, a occuparsi dello scambio dei messaggi.

Context unico per Sprint1

`cargoservice` e `ioport` sono stati mantenuti all'interno dello stesso `Context ctxcargoservice` , scelta di semplificazione esplicitamente temporanea: la separazione dei Context per nodo fisico è demandata allo Sprint2.

Test plans

Per lo Sprint1 ci concentriamo sul verificare che, a fronte di un input noto (un messaggio inviato all'attore), il sistema produca l'output atteso (una reply o un aggiornamento del display), indipendentemente da come è implementato internamente. Ogni caso di test corrisponde quindi a una sequenza osservabile di messaggi qak,

verificabile leggendo la reply ricevuta da `cargoservice` o il payload broadcast da `IOPortWsAdapter` verso i client WebSocket.

Caso	Precondizione	Azione	Risultato atteso
Richiesta accettata	almeno uno slot <code>FREE</code> in <code>hold</code> ; sensore non occupato	<code>pushButton</code> su <code>ioport</code>	<code>cargoservice</code> risponde <code>loadaccepted(SLOT,HOLD)</code> con uno <code>SLOT</code> valido; transita da <code>idle</code> a <code>engaged</code> ; il display mostra "Richiesta accettata – slot X"
Rifiuto: <code>retrylater</code>	sensore forzato a occupato (<code>setOccupied(true)</code>)	<code>pushButton</code>	<code>loadrejected(retrylater,HOLD)</code> ; il display mostra "Richiesta rifiutata: retrylater"
Rifiuto: full	tutti e 4 gli slot già <code>RESERVED</code> / <code>OCCUPIED</code> (4 richieste precedenti accettate)	<code>pushButton</code> con sensore libero	<code>loadrejected(full,HOLD)</code>
Timeout dei 30 s	richiesta appena accettata, <code>cargoservice</code> nello stato <code>engaged</code>	nessuna interazione per 30000 ms (<code>Transition t1 whenTime</code>)	<code>cargoservice</code> rilascia lo slot (<code>hold.releaseReserved()</code>), torna <code>idle</code> e invia a <code>ioport</code> un <code>updateDisplay</code> con stato <code>IDLE</code> e messaggio "Timeout scaduto: slot liberato, torno idle"

Trattandosi di un sistema che espone la propria interazione tramite un pannello GUI, i casi sono al momento eseguibili manualmente dal pannello di controllo, osservando le reply e i log colorati prodotti dagli attori.

Project

Il sistema è descritto interamente da [sprint1.qak](#), unica fonte da cui il toolchain qak genera il codice Kotlin degli attori; le classi in `it.unibo.cargoservice` / `it.unibo.ioport` non sono scritte a mano.

sprint1.qak

```
Request loadrequest : loadrequest(OCCUPIED)
Reply  loadaccepted : loadaccepted(SLOT,HOLD) for loadrequest
Reply  loadrejected : loadrejected(REASON,HOLD) for loadrequest

Dispatch pushButton : pushButton(NONE)
Dispatch setOccupied : setOccupied(FLAG)
Dispatch updateDisplay : updateDisplay(STATE,HOLD,MSG)

Context ctxcargoservice ip [host="localhost" port=8050]
```

`loadrequest` è la request/reply tra `ioport` e `cargoservice` ; `pushButton` e `setOccupied` sono i dispatch che l'adapter inietta in `ioport` per conto del browser; `updateDisplay` è l'unico dispatch con cui `cargoservice` comunica a `ioport` un aggiornamento da mostrare, qualunque ne sia l'origine.

cargoservice

Incapsula la `Hold` tramite `withobj hold using "utils.cargoservice.Hold()"` . Alla ricezione di `loadrequest` : rifiuta `retrylater` se occupato, altrimenti tenta `hold.reserveFirstFree()` e se riesce risponde `loadaccepted` . `Transition t1 whenTime 30000 -> disengaged` è il timeout che rilascia lo slot e inoltra esplicitamente a `ioport` un `updateDisplay` ; il timeout è un'iniziativa autonoma (dispatch) di `cargoservice` , non una risposta (reply)a qualcosa che `ioport` ha chiesto.

sprint1.qak

```
State handleRequest {
    onMsg(loadrequest : loadrequest(X)) {
        [# Occupied = payloadArg(0).toBoolean() #]
        if [# Occupied == true #] {
```

```

        replyTo loadrequest with loadrejected : loadrejected(retrylater,$H)
    } else {
        [# Slot = hold.reserveFirstFree() #]
        if [# Slot != -1 #] {
            replyTo loadrequest with loadaccepted : loadaccepted($Slot,$H2)
        } else {
            replyTo loadrequest with loadrejected : loadrejected(full,$H)
        }
    }
}

Goto engaged if [# Slot != -1 && Occupied == false #] else idle

State disengaged {
    [# hold.releaseReserved() #]
    [# var S = "'IDLE'" #]
    [# var M = "'Timeout scaduto: slot liberato, turno idle'" #]
    forward ioport -m updateDisplay : updateDisplay($S,$H,$M)
    [# Slot = -1 #]
}

Goto idle

```

Il LED fisico non è ancora un'interazione qak modellata: nei due punti in cui andrebbe acceso/spento, il codice si limita per ora a un `println("LED PicoW -> on/off")`, in attesa del microcontrollore reale che verrà sviluppato nello Sprint2.

ioport

Reagisce a tre soli dispatch (`pushButton` , `setOccupied` , `updateDisplay`), oltre alle reply di `cargoservice` . Il flag di occupazione è incapsulato in `IOPortStatus.java`. Alla pressione del pulsante genera la `loadrequest` leggendo quel campo:

```

State onButtonPressed {
    onMsg(pushButton : pushButton(X)) {
        [# var Occ = status.isOccupied() #]
        request cargoservice -m loadrequest : loadrequest($Occ)
    }
}

```

[sprint1.qak](#)

Ricevuta la reply, costruisce lei stessa il messaggio (stato) da mostrare e chiama l'adapter.

```

State showAccepted {
    onMsg(loadaccepted : loadaccepted(SLOT,HOLD)) {
        [# WsAdapter.updateDisplay("ENGAGED", H, "Richiesta accettata - slot " + S) #]
    }
}

Goto idle

State onUpdateDisplay {
    onMsg(updateDisplay : updateDisplay(STATE,HOLD,MSG)) {
        [# WsAdapter.updateDisplay(S, H, M) #]
    }
}

Goto idle

```

[sprint1.qak](#)

L'attore istanzia l'adapter nel proprio stato iniziale, passandogli un riferimento a se stesso e la porta HTTP:

```

QActor ioport context ctxcargoservice
    withobj status using "utils.cargoservice.IOPortStatus()" {

```

[sprint1.qak](#)


```
[# var WsAdapter = utils.cargoservice.IOPortWsAdapter(this, 7070) #]
```

IOPortWsAdapter

Il costruttore avvia un server Javalin che serve la GUI statica ed espone l'endpoint `/ws/ioport`, visualizzabile da tutti i clienti attraverso un broadcast:

[IOPortWsAdapter.java](#)

```
app.ws("/ws/ioport", ws -> {
    ws.onConnect(ctx -> clients.add(ctx));
    ws.onClose(ctx -> clients.remove(ctx));
    ws.onMessage(ctx -> handleClientMessage(ctx.message()));
});
```

Per quanto riguarda il formato dei messaggi condivisi, invece del formato nativo di `IAppMessage`, si è scelto un envelope JSON minimale con un solo campo discriminante `type`, motivato dalla leggerezza e dal parsing nativo in JavaScript (`JSON.parse`, senza librerie). Questa scelta implementativa rende necessario il passo di mappatura verso il dispatch qak, che `toDispatch` esegue:

[IOPortWsAdapter.java](#)

```
static String[] toDispatch(String raw) throws Exception {
    JSONObject json = (JSONObject) new JSONParser().parse(raw);
    String type = (String) json.get("type");
    if ("pushButton".equals(type)) {
        return new String[] { "pushButton", "pushButton(0)" };
    } else if ("setOccupied".equals(type)) {
        boolean value = Boolean.TRUE.equals(json.get("value"));
        return new String[] { "setOccupied", "setOccupied(" + value + ")" };
    }
    throw new IllegalArgumentException("unknown message type: " + type);
}

private void sendToActor(String msgId, String content) {
    IAppMessage dispatch = CommUtils.buildDispatch(SENDER, msgId, content, IOPORT_NAME);
    ioport.sendMsgToMyself(dispatch);
}
```

Nella direzione opposta non c'è questo problema: `updateDisplay` / `notifySensor` sono chiamate dirette e sincrone da `ioport` (è l'attore a tenere l'iniziativa dal proprio thread), ciascuna trasmessa in broadcast a tutti i client connessi.

GUI

[ioport_gui.html](#) e [wscontrol.js](#) non contengono logica di business: inviano i due soli comandi previsti, e aggiornano il DOM in base al tipo di messaggio ricevuto in broadcast.

[wscontrol.js](#)

```
btn.addEventListener('click', () => {
    socket.send(JSON.stringify({ type: 'pushButton' }));
    startTimer();
});
```

Il countdown avviato da `startTimer()` è puramente rappresentativo: disabilita il pulsante lato client, ma non è la fonte di verità del timeout, gestito interamente lato server dalla `Transition t1 whenTime 30000` vista sopra. Un client disconnesso o in ritardo non può quindi disallineare lo stato reale del sistema; alla riconnessione (ritentata ogni 2 s) riparte da uno stato neutro finché non arriva il prossimo `display` in broadcast.

Deployment

Il progetto si avvia dalla cartella `sprint1` tramite `gradlew run`

Il sistema espone due porte distinte, con responsabilità separate:

Porta	Componente	Funzione
8050	<code>Context ctxcargoservice</code>	porta di infrastruttura del Context qak, riservata all'eventuale comunicazione remota tra attori; non utilizzata nello Sprint1 poiché <code>cargoservice</code> e <code>ioport</code> condividono lo stesso Context
7070	Server Javalin embedded (<code>IOPortWsAdapter</code> , istanziato dall'attore <code>ioport</code>)	traffico HTTP/WS della GUI: redirect da <code>/</code> a <code>/ioport_gui.html</code> , endpoint WebSocket su <code>/ws/ioport</code>

Il pannello di controllo è raggiungibile all'indirizzo `http://localhost:7070/ioport_gui.html` a sistema avviato.



Definizione dello Sprint2

Lo Sprint2 dovrà:

REQUISITI

- Progettare e realizzare l'attore `cargorobot` , che orchestra il movimento del DDR appoggiandosi al servizio esterno `RobotSmart26` : raggiungere l'IOPort, prelevare il container, passare da `slot5` (marcatura) allo slot riservato, tornare in HOME.
- Modellare il `marker` : segnalazione (interna al nodo `cargoservice`) del completamento della marcatura in `slot5` , prima che il robot possa prelevare il container.
- Separare i Context per nodo fisico (`cargoservice` , `cargorobot` , `ioport`), oggi ancora unificati in `ctxcargoservice` per semplicità.



Valeria Capacci
valeria.capacci@studio.unibo.it



Ginevra Pentoli
ginevra.pentoli@studio.unibo.it

GIT: github.com/valecapa/TemaFinaleISS2026